

SIMATS ENGINEERING

AI-Powered Smart Water Reservoir Management and Demand Forecasting Platform

**COURSE NAME: SOFTWARE
ENGINEERING**

COURSE CODE: CSA1013

**Submitted by:
VISHAL B
192524485
B.TECH AIDS**

1. Problem Analysis

1.1 Problem Statement

Traditional reservoir management relies heavily on historical observation and manual, experience-based decision-making. This approach becomes unreliable under climate variability, unpredictable rainfall, and growing population demand, frequently resulting in inefficient water allocation during droughts as well as during periods of heavy rainfall. The AI-Powered Smart Water Reservoir Management and Demand Forecasting Platform addresses this gap by continuously monitoring reservoir levels, rainfall forecasts, inflow and outflow rates, and consumption patterns, and by using this data to predict future water availability, recommend optimal distribution strategies, and support authorities in sustainable, evidence-based water resource planning.

1.2 Project Objectives

- Monitor reservoir water levels in real time using IoT-based sensors.
- Forecast short-term and seasonal water demand using historical consumption and AI models.
- Predict future water availability by combining rainfall, inflow, and outflow data.
- Optimize water distribution across drinking, irrigation, and industrial consumers.
- Analyze rainfall and satellite meteorological patterns for early warning.
- Generate automated water management and utilization reports for authorities.
- Support drought planning and contingency scenario modelling.
- Improve overall water resource utilization and reduce wastage.

1.3 Functional Requirements

- Ingestion of real-time sensor data (water level, flow rate, rainfall gauges) via IoT gateways.
- Integration with satellite/weather APIs for rainfall and climate forecasts.
- An AI/ML forecasting service that predicts demand and inflow using time-series models.
- A rule/optimization engine that recommends distribution strategies across zones.

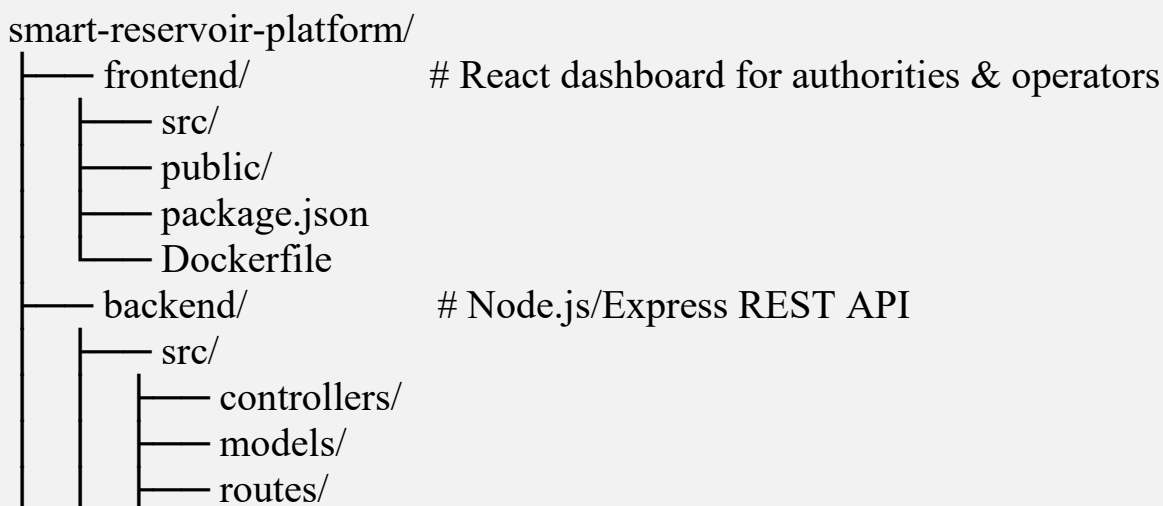
- A web-based dashboard for authorities to visualize reservoir status and reports.
- Alerting and notification module for drought or flood-risk thresholds.
- Secure role-based access for administrators, field engineers, and policy makers.
- A reporting module capable of exporting periodic water management reports.

1.4 Expected Outcomes

- Efficient and equitable water allocation across sectors.
- Improved day-to-day reservoir management with reduced manual intervention.
- Reduced incidence of water shortages through early prediction.
- Better preparedness for drought and extreme-rainfall events.
- Sustainable, data-driven water utilization.
- Higher accuracy in demand forecasting compared to manual estimation.
- Enhanced environmental conservation through optimized release schedules.
- Improved reliability of public water supply.

2. Project Structure Design

The repository follows a modular, service-oriented folder structure so that the frontend dashboard, backend APIs, AI forecasting engine, IoT data ingestion service, and deployment configuration can be developed, tested, and containerized independently while still being orchestrated together through Docker Compose.



```

├── middleware/
├── package.json
├── Dockerfile
├── ai-forecasting-service/  # Python ML microservice
│   ├── models/           # trained demand/inflow forecasting models
│   ├── app.py             # Flask/FastAPI inference service
│   ├── requirements.txt
│   └── Dockerfile
├── iot-ingestion-service/  # Sensor & rainfall data ingestion
│   ├── src/
│   ├── package.json
│   └── Dockerfile
├── database/
│   ├── init.sql           # schema for reservoirs, sensors, forecasts
│   └── migrations/
├── docs/                  # architecture & workflow diagrams
├── screenshots/           # Git & Docker execution evidence
├── .github/               # CI/CD workflow definitions
├── .gitignore
├── docker-compose.yml
├── .env.example
└── README.md

```

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
frontend/	React-based dashboard used by authorities to view reservoir status, forecasts and reports.
backend/	Core REST API handling authentication, reservoir CRUD, and orchestration of other services.
ai-forecasting-service/	Python microservice hosting trained ML models for demand and inflow prediction.
iot-ingestion-service/	Receives and normalizes real-time sensor and rainfall gauge data.

Folder / File	Purpose
database/init.sql	Defines initial schema — reservoirs, sensor readings, forecasts, users.
docs/	Architecture, ER, and branching diagrams referenced in this report.
.gitignore	Excludes node_modules, __pycache__, .env, and build artifacts from version control.
docker-compose.yml	Defines and orchestrates all containers, networks, and volumes for local/staging runs.
.env.example	Template for required environment variables without exposing secrets.
README.md	Setup instructions, architecture overview, and usage guide.

3. Git Repository Initialization

3.1 Initialization Steps

```
$ mkdir smart-reservoir-platform && cd smart-reservoir-platform
$ git init
$ git config user.name "Your Name"
$ git config user.email "your.email@example.com"
$ touch README.md .gitignore
$ git add README.md .gitignore
$ git commit -m "chore: initial commit with README and gitignore"
$ git branch -M main
$ git remote add origin <repository-url>
$ git push -u origin main
```

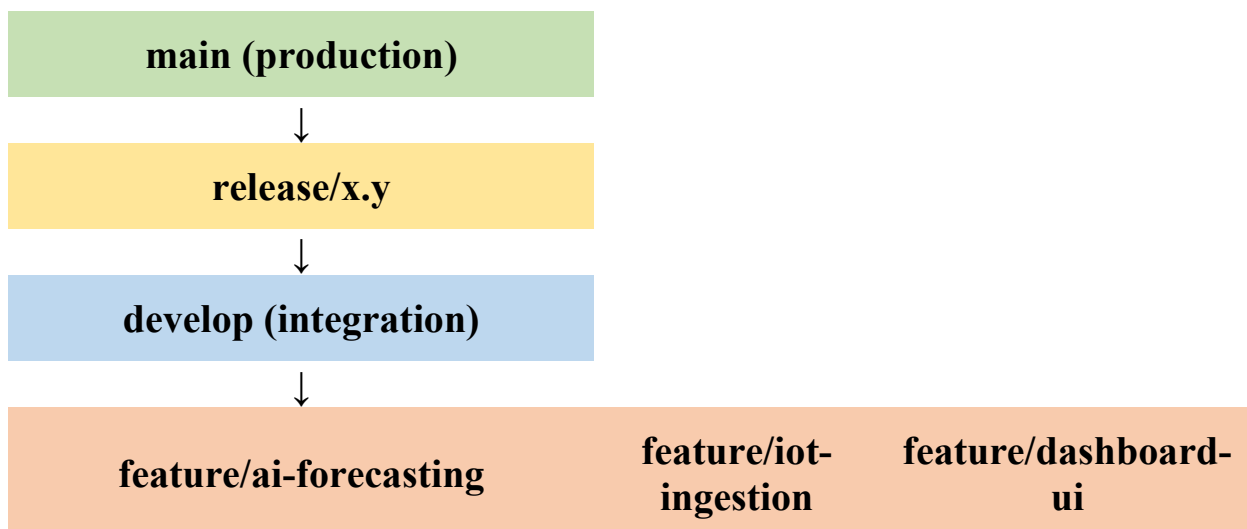
3.2 Purpose of Essential Git Files

File / Folder	Purpose
.git/	Hidden directory created by `git init`; stores the object database, refs, and repository metadata.
.gitignore	Lists files/folders (node_modules, __pycache__, .env, *.log) that Git should not track.
README.md	Documents project purpose, setup steps, and usage for collaborators.
.github/workflows/	Holds CI/CD pipeline definitions for automated build, test, and Docker image publishing.

Repository initialization establishes the local .git metadata store, links the working directory to a remote (e.g., GitHub/GitLab), and records the first baseline commit. This baseline gives every subsequent branch, feature, and release a common, traceable starting point.

4. Git Branching Strategy

The project adopts a Gitflow-inspired branching model, which is well suited to a multi-service platform (frontend, backend, AI service, IoT ingestion) with periodic releases to authorities and continuous feature development by multiple team members.



hotfix/* branches off main → merged into main & develop

4.1 Branch Roles

Branch	Purpose
main	Always deployment-ready; reflects the current production state used by reservoir authorities.
develop	Integration branch where completed features are merged and tested together.
feature/*	Isolated work for a single capability, e.g. feature/ai-forecasting, feature/iot-ingestion.
release/*	Stabilization branch created from develop before a versioned deployment; only bug fixes allowed.
hotfix/*	Urgent production fixes branched from main and merged back into both main and develop.

4.2 Justification

- Multiple independent services (frontend, backend, AI, IoT) are developed in parallel — feature branches prevent interference between teams.
- main must remain stable since it drives a platform used for real-world drought and supply decisions; Gitflow keeps unfinished work out of main.
- release/* branches allow final testing of forecasting accuracy and containerized deployment before authorities rely on the data.
- hotfix/* supports rapid correction of critical issues such as an incorrect drought alert without waiting for the next release cycle.

5. Version Control Workflow

The following sequence demonstrates a realistic development cycle for the AI forecasting feature, including branch creation, commits, a merge conflict and its resolution, and synchronization with the remote repository.

```
# 1. Create and switch to a feature branch
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/ai-forecasting

# 2. Make meaningful, incremental commits
$ git add ai-forecasting-service/app.py
$ git commit -m "feat(ai): add LSTM-based demand forecasting endpoint"
$ git add ai-forecasting-service/models/inflow_model.pkl
$ git commit -m "feat(ai): integrate trained inflow prediction model"

# 3. Keep the branch up to date and handle conflicts
$ git fetch origin
$ git merge origin/develop
Auto-merging ai-forecasting-service/app.py
CONFLICT (content): Merge conflict in ai-forecasting-service/app.py
$ git diff          # inspect conflicting hunks
# edit app.py to reconcile both changes
$ git add ai-forecasting-service/app.py
$ git commit -m "fix: resolve merge conflict in forecasting endpoint"

# 4. Push feature branch and open a Pull Request
$ git push -u origin feature/ai-forecasting

# 5. Merge into develop after review
$ git checkout develop
$ git merge --no-ff feature/ai-forecasting
$ git push origin develop
```


5.1 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates an isolated branch so forecasting work does not affect other features.
git add / commit	Records a meaningful, atomic snapshot of a completed unit of work.
git fetch + merge	Synchronizes the feature branch with the latest integration branch.
Conflict resolution	Manually reconciles overlapping edits so both contributions are preserved.
git push -u	Publishes the branch to the remote and sets upstream tracking.
git merge --no-ff	Merges into develop while preserving the feature's commit history as a distinct branch.

6. Commit History Analysis

6.1 Sample Commit History

```
$ git log --oneline --graph --decorate
* a91f3c2 (HEAD -> develop) Merge branch 'feature/ai-forecasting' into
develop
|
| * 7d2e6b1 fix: resolve merge conflict in forecasting endpoint
| * 4c8a9f0 feat(ai): integrate trained inflow prediction model
| * 1b3d5e7 feat(ai): add LSTM-based demand forecasting endpoint
|/
* e6f1a08 feat(iot): add rainfall gauge data ingestion service
* c2b7d34 feat(backend): implement reservoir CRUD API
* 9a4e2c1 feat(frontend): add reservoir status dashboard skeleton
* 5f0d8b3 chore: configure docker-compose for local development
* 3e1c9a2 chore: initial commit with README and gitignore
```

6.2 Commit Message Practices

- Conventional prefixes (feat, fix, chore) immediately convey the type of change.
- Each message names the affected module (ai, backend, iot, frontend) for quick traceability.
- Messages describe intent ("add LSTM-based demand forecasting endpoint") rather than restating the diff.
- Commits are atomic — one logical change per commit — making git bisect and code review straightforward.
- This discipline simplifies release note generation and helps new contributors understand project evolution without reading full diffs.

7. Docker Environment Design

7.1 Why Docker Is Suitable

The platform integrates several heterogeneous components — a Node.js backend, a Python-based AI forecasting engine, a React frontend, an IoT ingestion service, and a database — each with different runtime dependencies. Docker packages every component with its exact dependencies into a portable image, guaranteeing that the platform behaves identically on a developer's laptop, a testing server, and the production infrastructure used by water authorities. This eliminates "works on my machine" issues and allows each service to be scaled or restarted independently.

7.2 Components Requiring Containerization

Component	Reason for Containerization
frontend (React)	Isolated Node build/runtime for the dashboard, served via Nginx.
backend (Node/Express API)	Core API with its own Node.js version and npm dependencies.
ai-forecasting-service (Python)	Requires specific Python, TensorFlow/PyTorch, and ML library versions.
iot-ingestion-service	Handles continuous sensor streams; benefits from independent scaling.

Component	Reason for Containerization
PostgreSQL database	Persistent storage for reservoir, sensor, and forecast data.
Redis (optional cache)	Caches frequent forecast queries to reduce database load.

8. Dockerfile Development

Below is the Dockerfile for the AI forecasting microservice, chosen as the representative example since it has the most complex dependency set.

```
# ai-forecasting-service/Dockerfile
FROM python:3.11-slim AS base

# Set working directory inside the container
WORKDIR /app

# Install OS-level dependencies required by ML libraries
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev && rm -rf /var/lib/apt/lists/*

# Copy dependency manifest first to leverage Docker layer caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source and trained models
COPY . .

# Create a non-root user for security
RUN useradd -m appuser && chown -R appuser /app
USER appuser

# Document the service port
EXPOSE 5000

# Health check for orchestration
HEALTHCHECK --interval=30s --timeout=5s CMD curl -f
http://localhost:5000/health || exit 1
```

```
# Start the inference service
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

8.1 Purpose of Each Instruction

Instruction	Purpose
FROM python:3.11-slim	Selects a lightweight, reproducible Python base image.
WORKDIR /app	Sets the container's working directory for subsequent commands.
RUN apt-get install	Installs OS packages needed to compile ML/DB client libraries.
COPY requirements.txt	Copied separately so pip install is cached unless dependencies change.
RUN pip install	Installs Python dependencies (Flask/FastAPI, scikit-learn, TensorFlow, etc.).
COPY . .	Copies application code and trained model files into the image.
USER appuser	Runs the container as a non-root user to reduce security risk.
EXPOSE 5000	Documents the port the forecasting API listens on.
HEALTHCHECK	Lets Docker/Compose detect and restart an unresponsive service.
CMD	Defines the default command that starts the Gunicorn-served API.

9. Docker Compose Design

docker-compose.yml orchestrates all services — frontend, backend, AI forecasting engine, IoT ingestion, PostgreSQL, and Redis — as a single, reproducible multi-container application.

```
# docker-compose.yml
version: "3.9"

services:
  frontend:
    build: ./frontend
    ports: ["3000:80"]
    depends_on: [backend]
    networks: [reservoir-net]

  backend:
    build: ./backend
    ports: ["4000:4000"]
    environment:
      - DB_HOST=database
      - AI_SERVICE_URL=http://ai-forecasting:5000
    env_file: .env
    depends_on: [database, ai-forecasting]
    networks: [reservoir-net]

  ai-forecasting:
    build: ./ai-forecasting-service
    ports: ["5000:5000"]
    volumes:
      - ./ai-forecasting-service/models:/app/models
    networks: [reservoir-net]

  iot-ingestion:
    build: ./iot-ingestion-service
    ports: ["6000:6000"]
    depends_on: [database]
    networks: [reservoir-net]

  database:
    image: postgres:15
    environment:
      - POSTGRES_DB=reservoir_db
      - POSTGRES_USER=admin
```

```
- POSTGRES_PASSWORD=${DB_PASSWORD}
volumes:
- db-data:/var/lib/postgresql/data
- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
networks: [reservoir-net]
```

```
redis:
  image: redis:7-alpine
  networks: [reservoir-net]
```

```
volumes:
  db-data:

networks:
  reservoir-net:
    driver: bridge
```

9.1 Services, Networking, Volumes, and Dependencies

Aspect	Explanation
Services	Six services: frontend, backend, ai-forecasting, iot-ingestion, database, redis — each built from its own Dockerfile or a public image.
Networking	A dedicated bridge network reservoir-net lets containers reach each other by service name (e.g. <code>http://ai-forecasting:5000</code>).
Volumes	db-data persists PostgreSQL data across restarts; a bind mount exposes trained models to the AI service without rebuilding the image.
Environment Variables	Secrets such as DB_PASSWORD are supplied via <code>.env</code> and never hard-coded in the compose file.
Dependencies (<code>depends_on</code>)	Ensures the backend starts only after the database and AI service are

Aspect	Explanation
	available, and the frontend waits for the backend.

10. Container Deployment and Testing

10.1 Execution Commands

```
$ docker compose build
$ docker compose up -d
$ docker compose ps
```

NAME	STATUS	PORTS
reservoir-frontend	Up 2 minutes	0.0.0.0:3000->80/tcp
reservoir-backend	Up 2 minutes	0.0.0.0:4000->4000/tcp
reservoir-ai-forecasting	Up 2 minutes	0.0.0.0:5000->5000/tcp
reservoir-iot-ingestion	Up 2 minutes	0.0.0.0:6000->6000/tcp
reservoir-database	Up 2 minutes	0.0.0.0:5432->5432/tcp (healthy)
reservoir-redis	Up 2 minutes	6379/tcp

10.2 Testing Procedure

- Verified container health with `docker compose ps` and the `HEALTHCHECK` status for the AI service.
- Called `GET /health` on the backend and AI service to confirm both respond with HTTP 200.
- Sent a sample `POST /forecast/demand` request with historical consumption data and validated the returned forecast against expected ranges.
- Simulated a sensor reading through the IoT ingestion endpoint and confirmed it was persisted in PostgreSQL via `docker exec psql` queries.
- Loaded the React dashboard at `localhost:3000` and confirmed reservoir levels and forecast charts rendered correctly.
- Stopped and restarted the database container to confirm data persisted through the `db-data` volume.

Together these checks provided evidence that all containers start correctly, communicate over the `reservoir-net` network, and the platform performs its core

function — ingesting reservoir data and returning accurate forecasts — inside the containerized environment.

11. Benefits of Git and Docker

Dimension	Benefit for This Project
Collaboration	Feature branches let frontend, backend, AI, and IoT teams work in parallel without blocking each other; pull requests enable structured code review.
Version Management	Full commit history allows tracing exactly when a forecasting model or API change was introduced, easing debugging and audits.
Portability	Docker images run identically on a developer laptop, CI runner, or the government/authority's cloud infrastructure.
Deployment	docker compose up reproduces the entire multi-service stack in minutes, avoiding manual environment setup.
Scalability	Individual services (e.g. ai-forecasting) can be scaled independently during high-demand periods such as monsoon season.
Maintainability	Clear branch history and isolated containers make it easy to patch a single service (e.g. update the ML model) without redeploying the whole platform.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts arose when the AI forecasting and backend teams edited shared configuration files simultaneously, requiring careful manual reconciliation.
- Coordinating environment variables (database credentials, external weather-API keys) securely across multiple services without hard-coding them.
- Ensuring the AI forecasting container had correct startup ordering relative to the database and trained-model volume mounts.
- Managing image size for the AI service due to large machine-learning library dependencies, which slowed build and deployment time.
- Keeping documentation and commit messages consistent across contributors with varying Git experience.

12.2 Suggested Improvements and Future Enhancements

- Adopt a CI/CD pipeline (GitHub Actions) to automatically build, test, and push Docker images on every merge to develop.
- Use multi-stage Docker builds to shrink the AI service image and separate build-time dependencies from the runtime image.
- Introduce automated integration tests that spin up the full docker-compose stack in CI before merging feature branches.
- Add a secrets-management solution (e.g. Docker secrets or a vault service) instead of plain .env files for production deployments.
- Extend the branching strategy with mandatory pull-request templates and required reviewers to further standardize commit quality.
- Integrate container orchestration (Kubernetes) for auto-scaling the AI forecasting service during peak monsoon monitoring periods.